

B ve B+ Ağaçları

B -ağacı, sıralı verileri koruyan ve logaritmik sürede arama, sıralı erişim, ekleme ve silme işlemlerine izin veren, kendi kendini dengeleyen bir ağaç veri yapısıdır . B-ağacı, düğümlerin ikiden fazla çocuğa sahip olmasına izin vererek ikili arama ağacını geliştirir.

B ve B+ ağaçları, özellikle disk okuma/yazma (I/O) işlemlerinin maliyetli olduğu sistemlerde veriye en kısa yoldan ve logaritmik zaman karmaşıklığında ($O(\log n)$) ulaşmak için tasarlanmıştır.

Veri Tabanlarında Kullanımı

Veri tabanlarında temel amaç, devasa veri yığınları içinden istenilen kaydı diski en az yorarak bulmaktır.

İndeksleme Mekanizması: İlişkisel veri tabanlarında bir tabloya indeks eklendiğinde, sistem arka planda bir B+ ağacı oluşturur. Kök düğümden başlanarak yönlendirmelerle istenilen verinin bulunduğu disk bloğuna saniyelerden kısa sürede ulaşılır.

Aralık Sorguları: B+ ağaçlarında tüm veriler yaprak düğümlerde tutulur ve bu yapraklar birbirine bağlı bir liste (linked list) şeklindedir. Bu sayede "Maaşı 10.000 ile 20.000 arasında olan çalışanları getir" gibi sıralı okuma gerektiren aralık sorguları son derece hızlı çalışır.

Dosya Sistemlerinde Kullanımı

İşletim sistemleri, diskteki yüz binlerce dosyanın ve klasörün hiyerarşisini, izinlerini ve fiziksel disk üzerindeki konumlarını takip etmek zorundadır.

Dizin Yönetimi: Büyük bir klasörün içinde on binlerce dosya olduğunda, işletim sisteminin aradığınız bir dosyayı bulması için klasörü baştan sona taraması çok yavaştır. B-ağaçları sayesinde klasör içerikleri alfabetik veya hash sırasına göre indekslenir ve arama süresi minimize edilir.

Meta Veri (Metadata) Depolama: Dosya boyutları, oluşturulma tarihleri ve sahiplik gibi meta veriler B-ağaçları üzerinde tutularak dosya özelliklerine anında erişim sağlanır.

Boş Alan Yönetimi: Disk üzerindeki boş veya dolu blokların takibi için B-ağacı türevleri kullanılır.

Kullanılan Sistemler: Windows'un kullandığı NTFS dosya sistemi dizinleri yönetmek için doğrudan B-ağaçlarını kullanır. Linux dünyasındaki Ext4, XFS ve Btrfs (adını doğrudan B-Tree'den alır) ile Apple'ın APFS dosya sistemi bu yapıları temel alır.

Arama Motorları ve Büyük Veri Sistemlerinde Kullanımı

Tersinir İndeks (Inverted Index) Yönetimi: Arama motorları, bir kelimenin hangi web sayfalarında geçtiğini bulmak için tersinir indeksler kullanır. Bu kelimelerin (sözlüğün) diskte tutulması ve hızla aranması B-ağaçları ile sağlanır.

Bölümlenmiş Veri Erişimi: Dağıtık büyük veri sistemlerinde, verinin hangi sunucuda veya bölüntüde (partition) olduğunu belirten yönlendirme tabloları, hızlı erişim için bellekte B-ağaçları formatında tutulur.

Modern Sistemlerde Tercih Edilmesinin Sebebi

Disk ve Blok Mimarisiyle Uyum

B-ağacı ailesindeki her bir düğüm, tam olarak bir işletim sistemi disk bloğuna (genellikle 4 KB veya 8 KB) sığacak şekilde tasarlanır. Bu sayede, ağaç üzerinde bir düğümden diğerine geçerken yapılan tek bir disk okuma işlemiyle yüzlerce referans tek seferde belleğe alınır. Geleneksel İkili Arama Ağaçlarında (Binary Search Tree) her bir düğüm için ayrı bir disk okuması gerekebilir, bu da okuma maliyetini inanılmaz derecede artırır.

Geniş ve Sığ Yapı

B ve B+ ağaçları aşağıya doğru uzamaktan ziyade yana doğru genişleme eğilimindedir. Her bir düğüm yüzlerce alt düğüm barındırabilir (buna *fan-out* denir). Bu yüksek dallanma faktörü sayesinde ağaç çok "sığ" kalır.

Örnek: 100 dallanma faktörüne sahip bir B+ ağacında sadece 4 seviye inerek yüz milyonlarca kayda ulaşılabilir.

Bu da, milyarlarca satırlık bir tabloda aradığınız bir veriyi bulmak için devasa diske sadece 3 veya 4 kez başvurmanız anlamına gelir.

Öngörülebilir ve Kesin Performans

Kendi kendini dengeleyen yapılar oldukları için ekleme, silme ve arama işlemleri her zaman $O(\log n)$ zaman karmaşıklığında gerçekleşir. Veri tabanına sürekli yeni veriler eklense bile ağaç tek bir tarafa doğru uzayıp yozlaşmaz, her zaman dengede kalır. Bu da veri tabanının milyarlarca satırda bile arama süresini tahmin edilebilir, stabil bir seviyede tutmasını sağlar.

Veri Tabanı Sistemlerinde Kullanım Örnekleri

MySQL (InnoDB Depolama Motoru)

MySQL'in varsayılan ve en güçlü depolama motoru olan InnoDB, veri saklama ve indeksleme stratejisini tamamen **B+ Ağaçları** üzerine kurmuştur. MySQL'de B+ ağaçlarının kullanımı iki ana kategoriye ayrılır:

Kümelî İndeks (Clustered Index): MySQL'de bir tablo oluşturduğunuzda ve bir Birincil Anahtar (Primary Key) belirlediğinizde, tablonun kendisi aslında bir B+ ağacına dönüşür. Ağacın yaprak (leaf) düğümleri **satırdaki verinin ta kendisini** tutar. Yani veri tabanındaki tablonuz, fiziksel olarak diske Birincil Anahtara göre sıralanmış bir B+ ağacı olarak yazılır. Bu nedenle primary key üzerinden yapılan okumalar inanılmaz derecede hızlıdır.

İkincil İndeksler (Secondary Indexes): Başka bir sütuna (örneğin e-posta adresine) indeks attığınızda, MySQL yeni bir B+ ağacı oluşturur. Ancak bu yeni ağacın yapraklarında satırın verisi bulunmaz; onun yerine o veriye ait **Birincil Anahtar (Primary Key) değeri** bulunur.

Çalışma Mantığı: İkincil indeks üzerinden arama yaptığınızda, MySQL önce e-posta ağacında arama yapar, bulduğu Primary Key değerini alır, ardından tablonun ana B+ ağacına giderek asıl veriyi çeker (Buna *Bookmark Lookup* denir).

PostgreSQL

PostgreSQL, veri mimarisi açısından MySQL'den farklı bir yol izler. Varsayılan indeks yapısı resmi dökümanlarda "B-Tree" olarak geçse de, modern eşzamanlılık optimizasyonlarına (Lehman ve Yao algoritması) sahip, sağa doğru bağlantıları olan gelişmiş bir B+ ağacı türevidir.

Yığın (Heap) Mimarisi: PostgreSQL, tablo verilerini ağaçların içinde (Kümelî İndeks olarak) tutmaz. Veriler **Heap (Yığın)** adı verilen sırasız dosyalarda tutulur.

İndeks Ağaçlarının Yapısı: PostgreSQL'de bir sütuna indeks (ister primary key, ister normal indeks) oluşturduğunuzda kurulan B-Ağacının yaprak düğümlerinde veri değil, **TID (Tuple Identifier)** adı verilen işaretçiler (pointer) bulunur. Bu işaretçiler, verinin diskin hangi sayfasında (page) ve hangi satırında (offset) bulunduğunu gösteren bir nevi fiziksel adres koordinatlarıdır.

Çalışma Mantığı: PostgreSQL'de indeks üzerinden bir arama yaptığınızda, sistem B-Ağacında logaritmik hızda gezinerek ilgili verinin TID'sini (adresini) bulur. Sonra doğrudan Heap dosyasına giderek o adresteki veriyi okur.

Not: Bu mimari, ikincil indekslerin çok daha az yer kaplamasını sağlar çünkü MySQL'deki gibi Primary Key değerini çoğaltmak yerine doğrudan fiziksel adresi tutar.

MongoDB (WiredTiger Depolama Motoru)

İlişkisel olmayan (NoSQL) belge tabanlı bir sistem olan MongoDB, 3.2 sürümünden itibaren varsayılan olarak WiredTiger depolama motorunu kullanır. MongoDB verileri JSON (BSON) formatında tutsa da, arama ve disk kaydı işlemlerinde geleneksel sistemler gibi **B-Ağacı (B-Tree)** yapısını tercih eder.

Belge (Document) Depolama: MongoDB'deki her "Collection" (tablo karşılığı), arka planda bir B-ağacı olarak saklanır. Varsayılan olarak her belgenin otomatik atanan bir `_id` (Object ID) alanı vardır ve bu `_id` ağacın anahtarı (key) olarak kullanılır.

İndeksleme: İkincil indeksler (örneğin "kullanıcı_adi" alanına atılan indeks) yine ayrı bir B-ağacı olarak oluşturulur. MongoDB'nin B-ağacı yaprakları, belgenin fiziksel diskteki konumuna işaret eder.

WiredTiger'ın Optimizasyonu: MongoDB'nin kullandığı motor, verileri diske yazarken ve RAM'de (cache) tutarken farklı B-ağacı optimizasyonları kullanır. RAM üzerindeki ağaç düğümlerini sıkıştırılmamış halde hızlı işlem için tutarken, diske yazılan düğümleri blok blok sıkıştırarak I/O maliyetini düşürür.

Günlük hayat örneđi: Hastane Arşiv Sistemi

Hastanemizde milyonlarca hastanın dosyası var ve her dosyanın üzerinde bir "Hasta Kimlik Numarası (ID)" yazıyor. Bir doktor, acil bir durumda belirli bir hastanın dosyasını arşivden saniyeler içinde istemektedir.

B-Tree: "Karışık Arşiv Sistemi"

B-Ağacı (B-Tree) sistemini, hastanenin arşiv binasına benzetebiliriz. Bu binada hem yönlendirme tabelaları vardır hem de dosyaların kendisi.

Nasıl Çalışır? Arşiv binasından içeri girdiniz. Danışmada (Kök Düğüm) bir tablo var. Tablo diyor ki: *"10.000 numaralı dosya buradadır. 10.000'den küçükler için sol koridora, büyükler için sağ koridora gidin."* Eğer aradığınız dosya tam olarak 10.000 ise, dosyayı hemen danışmadan alıp çıkarsınız. Aramaya devam etmenize gerek kalmaz.

Avantajı: Şanslıysanız, aradığınız kaydı binanın çok derinlerine (yapraklara) inmeden üst katlarda bulabilirsiniz.

Sorunu (Aralık Sorgusu Kabusu): Doktor sizden *"10.000 ile 10.050 arasındaki tüm hastaların dosyalarını"* istedi diyelim. 10.000 numaralı dosyayı danışmadan alırsınız. Sonra sağ koridora gidersiniz, 10.025'i oradan bulursunuz. Sonra o koridordaki odalara tek tek girip diğerlerini ararsınız. Katlar, koridorlar ve odalar arasında sürekli bir **aşağı-yukarı zıplama** (ağaçta gezinme) yaparsınız. Bu çok yorucu ve zaman alıcıdır.

B+ Tree: "Modern ve Organize Arşiv Sistemi"

İşte B+ Ağaçlarının (B+ Tree) modern veri tabanlarında standart olmasının sebebi bu karmaşayı çözmesidir. Hastane yönetimi arşivi baştan aşağı yeniden tasarlar.

Yeni Kurallar:

Üst katlarda, koridorlarda veya danışmada (İç Düğümler) **asla** hasta dosyası (veri) bulundurulmaz. Buralarda sadece ve sadece **yönlendirme tabelaları** (indeksler) vardır.

Tüm fiziksel hasta dosyaları (veriler), binanın en alt katındaki (Yaprak Düğümler) devasa dosya dolaplarında tutulur.

En alt kattaki tüm dosya dolapları, yan yana dizilmiş ve birbirine **bir ray sistemiyle (Linked List)** bağlanmıştır.

Arama Süreci: Hasta 45.000'i arıyorsunuz. Danışmadaki tabelaya bakarsınız: *"40.000 - 50.000 arası 2. kata git"*. 2. kata çıkarsınız, oradaki tabela: *"45.000 için Bodrum Kat, 3. Dolaba in"* der. Bodrum kata inip dosyayı alırsınız. Her arama için mutlaka en alt kata inmek zorundasınızdır ama sistem tıkr tıkr işler, sürpriz yoktur.

Asıl Sihir (Aralık Sorgusu): Doktor sizden *"45.000 ile 50.000 arasındaki tüm dosyaları"* istedi. Tabelaları takip ederek doğrudan en alt kattaki 45.000 numaralı dosyayı bulursunuz. İşte sihir burada başlar: Diğer dosyaları bulmak için tabelalara tekrar bakmanıza veya üst katlara çıkmanıza gerek yoktur! Dolaplar birbirine rayla bağlı (Linked List) olduğu için, 45.000'i aldıktan sonra yana doğru yürüyerek 45.001, 45.002... diyerek 50.000'e kadar tüm dosyaları saniyeler içinde toplayıp çıkarsınız.

Bu Veri Yapıları Neden Önemlidir?

En yalın haliyle B ve B+ ağaçları, yazılımın donanımla barışmasını sağlayan yapılardır. Algoritma tasarlarken RAM üzerinde harikalar yaratabiliriz; diziler, bağlı listeler, karma tabloları (hash tables) kurabiliriz. Ancak iş kalıcı depolamaya geldiğinde fiziksel sınırlar devreye girer.

İşlemcilerimiz ve belleklerimiz inanılmaz hızlarda çalışırken, en modern M.2 NVMe SSD'ler bile onlara kıyasla çok yavaş kalır. Eğer sistem diski verimsiz okursa, işlemci sürekli verinin gelmesini bekler ve bir "I/O darboğazı" oluşur. B+ ağaçları, veriyi disk üzerinde bloklara tam oturacak şekilde yerleştirerek diskin mekanik veya elektronik okuma sınırlarını aşar. Bu yapılar olmasaydı, bugün donanımı verimli kullanan işletim sistemlerinden veya milisaniyeler içinde cevap veren arka uç (backend) servislerinden bahsetmemiz imkansız olurdu.

Büyük Veri Çağında Neden İhtiyaç Duyulmaktadır?

Büyük veri çağının temel paradoksu şudur: Veri yığını devasa boyutlara ulaşırken, o yığının içinden istenen veriyi çekmek için gereken süreye olan tahammülümüz sıfıra inmiştir.

Gigabaytların terabaytlara, petabaytlara ulaştığı günümüz sistemlerinde, doğru veri yapıları kullanmazsanız veriyi bulmak için baştan sona tarama (full table scan) yapmanız gerekir. B+ ağaçlarının en büyük gücü olan $O(\log n)$ karmaşıklığı burada parlar. Veri boyutu üstel olarak artsa bile, ağacın derinliği çok yavaş büyür. Milyarlarca kaydın olduğu bir sistemde aradığınızı bulmak için diske hala sadece 4 veya 5 kez başvurmanız yeterlidir. Büyük veri, ancak B+ ağaçları gibi yapılar sayesinde "yönetilebilir" ve "anamlı" bilgiye dönüşebilir.

Gelecekte de Kullanılmaya Devam Edecek mi?

Kesinlikle evet, ancak evrim geçirerek. B-ağacı ailesi 1970'lerde icat edildi ve yarım asırdır bilgisayar bilimlerinin temel taşlarından biri olarak kalmayı başardı.

Fakat ihtiyaçlar değişiyor. Sadece okumanın değil, saniyede milyonlarca verinin *yazılmasının* gerektiği büyük veri akışlarında (örneğin sensör verileri veya anlık log kayıtları), sırf yazma hızı için optimize edilmiş **LSM (Log-Structured Merge)** ağaçları ön plana çıkmaya başladı. Ayrıca RAM hızında çalışan ancak elektrik kesildiğinde veriyi silmeyen kalıcı bellek (NVRAM) teknolojileri geliştikçe, B+ ağaçlarının "blok" mantığında bazı yapısal değişiklikler olacaktır.

Yine de verinin düzenli ve sıralı bir şekilde tutulmasının gerektiği her senaryoda, işletim sistemlerinin çekirdeklerinde ve dosya dizinlerinde B+ ağaçları altın standart olmaya devam edecektir. Yazılım dünyasında bir yapının 50 yıl dayanması, mimari bir zorunluluk olduğunun en büyük kanıtıdır.